Lecture 2: DNN Basics (Continued)

CS 580B1: Trustworthy Machine Learning

Fall 2024

Acknowledgments

Course slides derived from material created by Rajeev Alur and Osbert Bastani at UPenn (link to their course)

Other relevant courses:

CS 329T at Stanford

CS 521 at UIUC

ECE1784H/CSC2559H at U Toronto

Course logistics

- Now recording on Echo 360
- Form teams by Aug 23, 11:59 PM
- Final project can also be done as a group (2-3 members)

Agenda

- DNN Basics
  - DNN Computational Model
  - DNN Training
    - Backpropagation
- PyTorch Basics

DNN Basics

- A simple feedforward neural network with 1 hidden layer:

$$f_\theta(x) \coloneqq W^{(2)}\left(g\left(W^{(1)}x + b^{(1)}\right)\right) + b^{(2)}$$

where

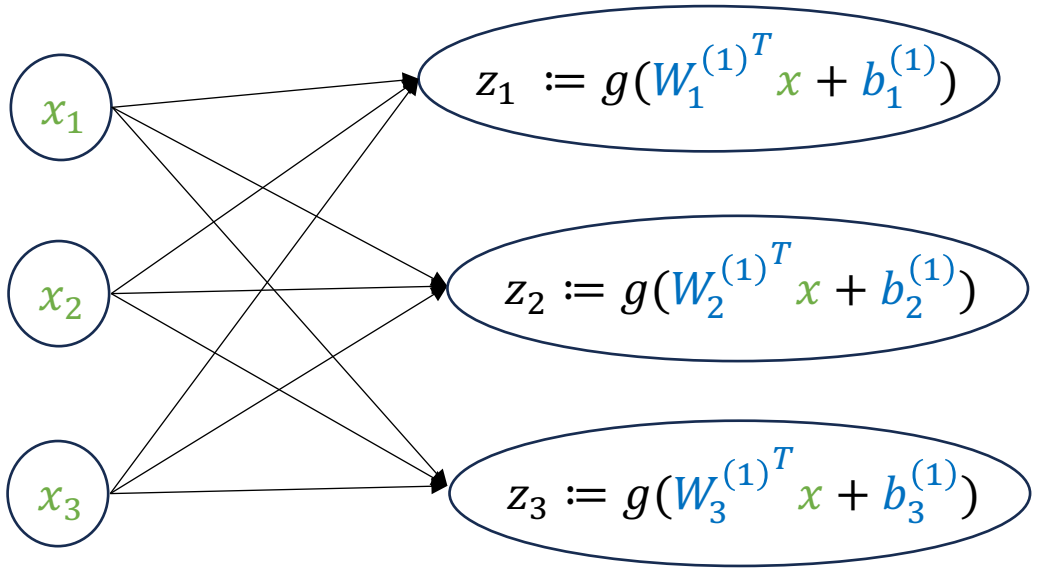
- $f_\theta : \mathbb{R}^i \to \mathbb{R}^o$,
- $W^{(1)} \in \mathbb{R}^{k\times i}, b^{(1)} \in \mathbb{R}^k, W^{(2)} \in \mathbb{R}^{o\times k}, b^{(2)} \in \mathbb{R}^o$ are parameters (or weights) to be learnt and k is the **number of hidden neurons**,
- $x \in \mathbb{R}^i$ is an input,
- $g : \mathbb{R} \to \mathbb{R}$ is an **activation function** applied component-wise, i.e., $g\left(\begin{bmatrix} z_1 \\ z_2 \end{bmatrix}\right) = \begin{bmatrix} g(z_1) \\ g(z_2) \end{bmatrix}$

Feedforward neural network with 1 hidden layer

$$f_\theta(x) \coloneqq x$$

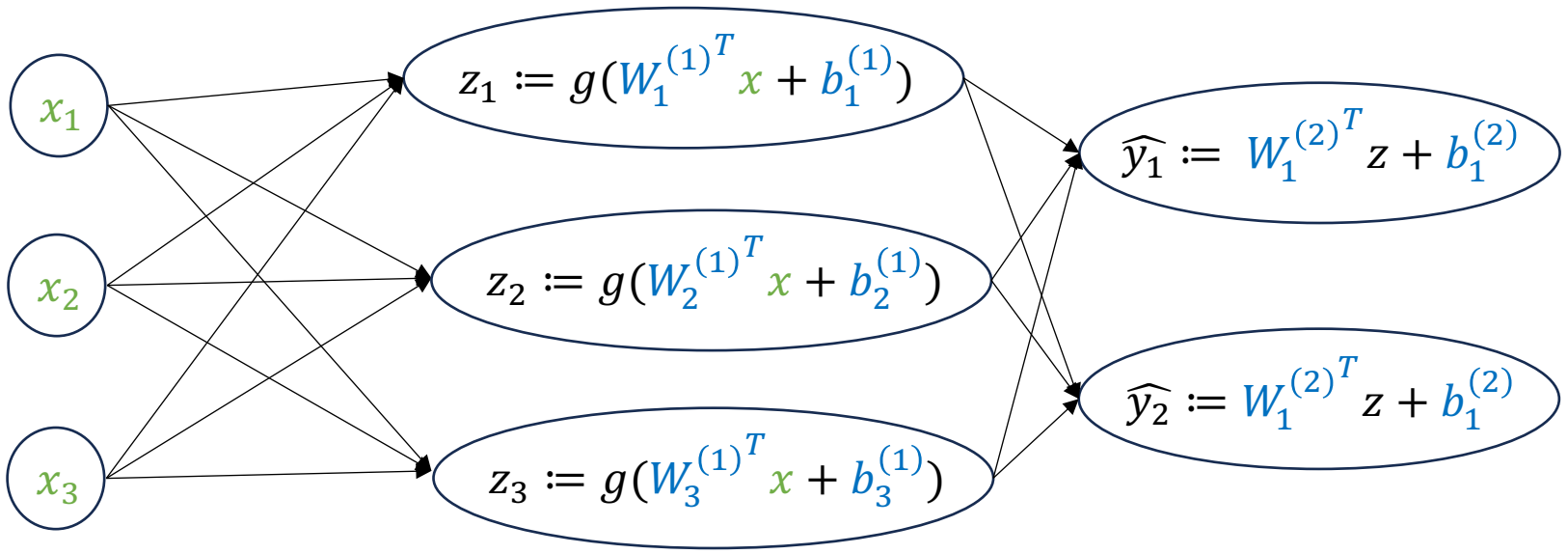
Feedforward neural network with 1 hidden layer

$$f_\theta(x) := g(W^{(1)}x + b^{(1)})$$



Feedforward neural network with 1 hidden layer

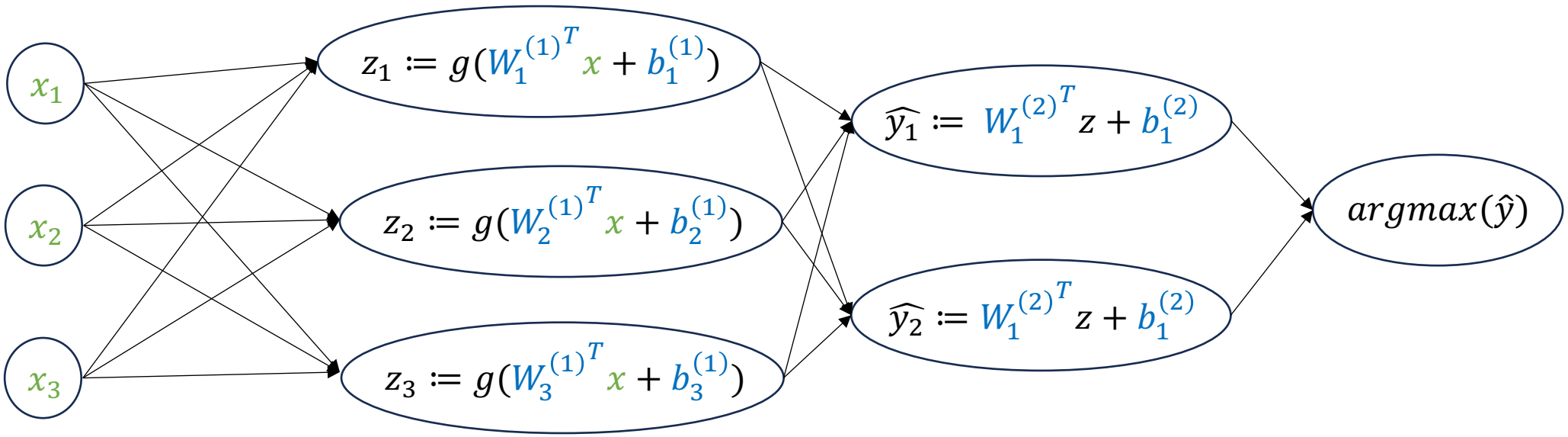
$$f_\theta(x) := W^{(2)}\left(g\left(W^{(1)}x + b^{(1)}\right)\right) + b^{(2)}$$



where $i = 3, o = 2, k = 3$

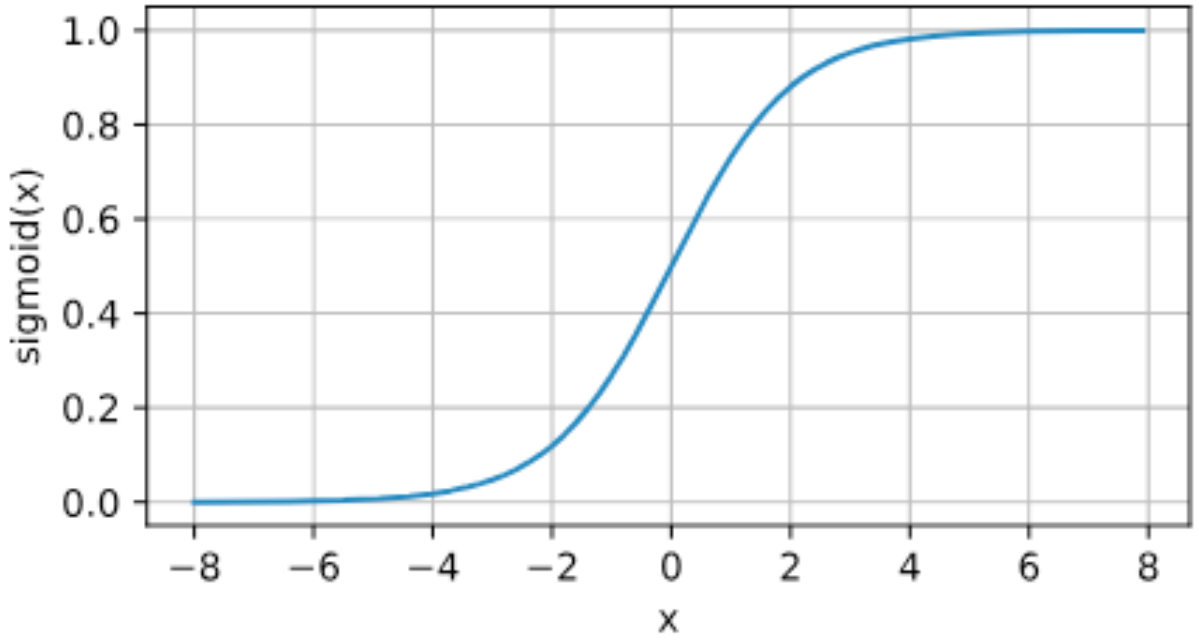
What about classification?

$$F_\theta(x) := argmax\big(f_\theta(x)\big) = argmax(W^{(2)}\Big(g\big(W^{(1)}x + b^{(1)}\big)\Big) + b^{(2)})$$



Activation functions

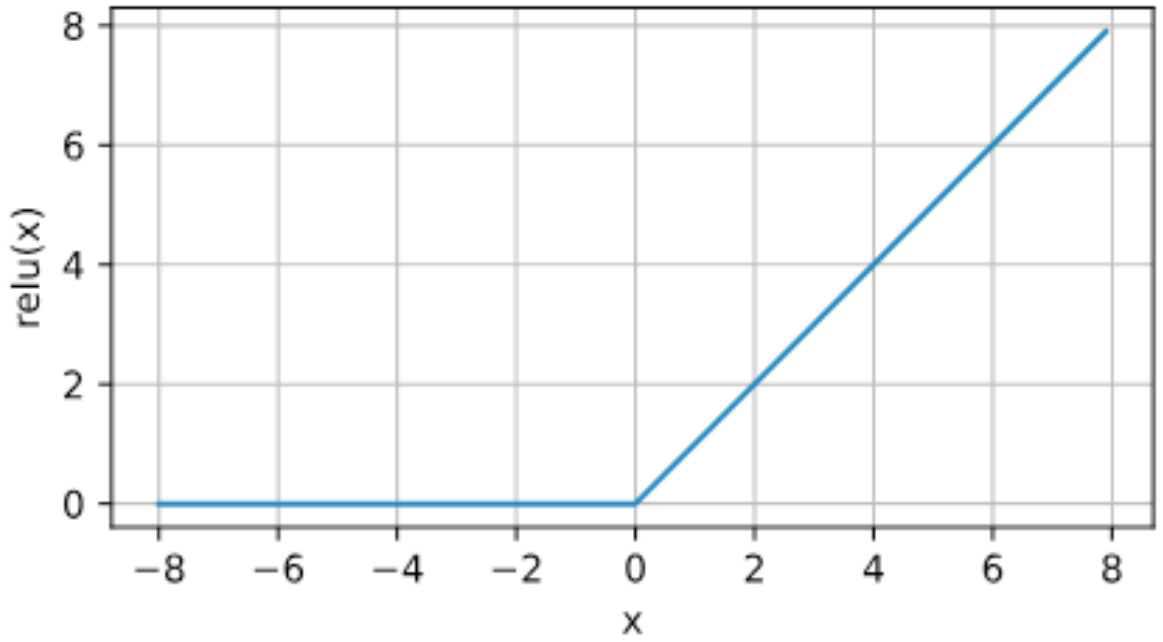
$$g(z) \coloneqq \sigma(z) \coloneqq \frac{1}{1+e^{-z}}$$



https://d2l.ai/chapter_multilayer-perceptrons/mlp.html

Activation functions

$$g(z) \coloneqq ReLU(z) \coloneqq \max(0, z)$$



https://d2l.ai/chapter_multilayer-perceptrons/mlp.html

Deep neural networks, generally

- "**Layers**" are building blocks
- Each layer is a parametric/non-parametric non-linear/linear function $f_{\theta(i)}: \mathbb{R}^{d'} \to \mathbb{R}^{d''}$
- Combine layers to form a model "**architecture**", for instance, sequential composition of affine (i.e., $Wz + b$) and activation layers to form a feed-forward DNN

$$f_\theta \coloneqq f_{\theta(m)}\left(\dots\left(f_{\theta(1)}\right)\dots\right) = f_{\theta(m)} \circ \cdots \circ f_{\theta(1)}$$

- Each layer is comprised of neurons/nodes/units

The deep learning recipe

- Collect large amounts of data, for instance, (x_i, y_i) pairs for supervised learning

- Design neural network architecture guided by "implicit" **structure/symmetries** in data (for instance, images are translation invariant, sentence should be read forwards)

- Learn model parameters by framing the learning problem as an optimization problem

Learning as optimization

Learning objective: Given model f_θ, find a valuation for parameters θ such that the **training loss** is minimized.

Training loss: For a training dataset with n samples (i.e., n pairs of (x_i, y_i)) and a model f_θ, the training loss is given by

$$\frac{1}{n}\sum_{i=1}^{n} L(f_{\theta_0}(x_i), y_i)$$

where $L(\widehat{y_i}, y_i)$ is a function that compares the output of the DNN on an input x_i with the ground-truth y_i.

- L can take many forms such as $(\hat{y}_i - y_i)^2$, $\mathbb{1}_{\widehat{y_i}=y_i}$, …

Learning as optimization

Learning objective: Given model f_θ, find a valuation for parameters θ such that the **training loss** is minimized.

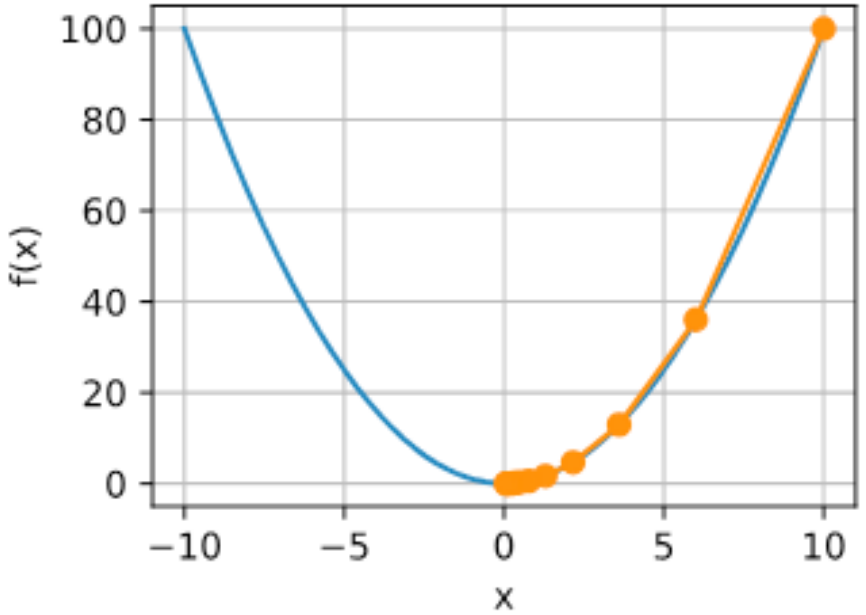
$$argmin_\theta \frac{1}{n} \sum_{i=1}^{n} L(f_\theta(x_i), y_i)$$

How to solve the optimization problem?

Gradient Descent!

Gradient descent algorithm

- Iterative algorithm to find the minimum of a function
- In each step, move in the direction opposite to the gradient
- Guaranteed to converge to the global minimum for **convex, differentiable** functions



https://d2l.ai/chapter_optimization/gd.html

Gradient descent for DNN training

$\theta_0 \leftarrow Initialize()$
$t \leftarrow 0$
$Until\ convergence$:
$\theta_{t+1}^{(j)} \leftarrow \theta_t^{(j)} - \eta \nabla_{\theta^{(j)}} \left(\frac{1}{n} \sum_{i=1}^{n} L\left(f_{\theta_t}(x_i), y_i\right) \right)$ (for each j)
$t \leftarrow t + 1$
$return\ f_{\theta_t}$

DNN training objectives are highly non-convex. Algorithm not guaranteed to converge!!

In practice, many small modifications to this algorithm to make it effective:

- Use stochastic gradient descent (SGD)
- How to initialize parameters?
- How to choose learning rate η? How to adapt it during training?
- Use of regularization such as weight decay
- Use of momentum

- See Chapter 12 of Dive into Deep Learning book and Chapter 8 of the Deep Learning book for more on DNN training algorithms

The deep learning recipe

- Collect large amounts of data, for instance, (x_i, y_i) pairs for supervised learning
- Design neural network architecture guided by "implicit" **structure/symmetries** in data (for instance, images are translation invariant, sentence should be read forwards)
- Learn model parameters via gradient descent
- Modern frameworks such as PyTorch, JAX, and TensorFlow make it very easy to define new architectures + come with in-built **generic** implementations of gradient descent

How to calculate the gradients?

Backpropagation!

Review of differentiation

Scalar functions

$$f: \mathbb{R} \to \mathbb{R}$$

$$\frac{df}{dx}: \mathbb{R} \to \mathbb{R}$$

Functions from vectors to scalars

$$f: \mathbb{R}^m \to \mathbb{R}$$

$$\text{(gradient) } \frac{df}{dx}: \mathbb{R}^m \to \mathbb{R}^{1\times m} = \nabla f \coloneqq [\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_m}]$$

Review of Differentiation

Functions from vectors to vectors

$$f: \mathbb{R}^m \to \mathbb{R}^n$$

(Jacobian) $\frac{df}{dx}: \mathbb{R}^m \to \mathbb{R}^{n \times m} = Jf = \nabla f :=$

$$\begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_m} \\ \vdots & & \\ \frac{\partial f_n}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_m} \end{bmatrix}$$

Backpropagation

$$F(x) \coloneqq L\overbrace{\left(f^{(2)}\left(f^{(1)}(x)\right)\right)}^{f}$$

$$\begin{aligned} F &: \mathbb{R}^d \to \mathbb{R} \\ f^{(1)} &: \mathbb{R}^d \to \mathbb{R}^{d'} \\ f^{(2)} &: \mathbb{R}^{d'} \to \mathbb{R}^{d''} \\ L &: \mathbb{R}^{d''} \to \mathbb{R} \end{aligned}$$

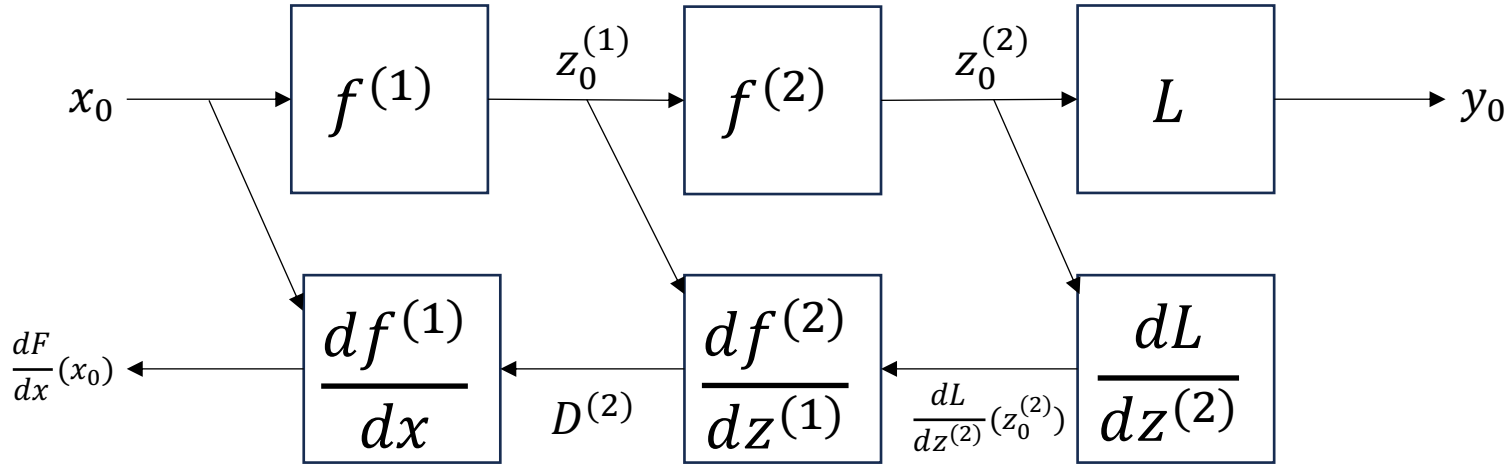
$$\begin{aligned} y &= F(x) \\ z^{(1)} &= f^{(1)}(x) \\ z^{(2)} &= f^{(2)}\left(z^{(1)}\right) \\ y &= L\left(z^{(2)}\right) \end{aligned}$$

$$\frac{dF}{dx} : \mathbb{R}^d \to \mathbb{R}^{1\times d} = \frac{dL}{dz^{(2)}}\frac{df^{(2)}}{dz^{(1)}}\frac{df^{(1)}}{dx} \text{ (chain rule)}$$

Backpropagation

How to calculate $\frac{dF}{dx}$ at a specific input x_0?

$$\frac{dF}{dx}(x_0) = \Bigg(\underbrace{\left(\frac{dL}{dz^{(2)}}\left(z_0^{(2)}\right) \frac{df^{(2)}}{dz^{(1)}}\left(z_0^{(1)}\right) \right)}_{D^{(2)}} \frac{df^{(1)}}{dx}(x_0) \Bigg)$$



Backpropagation for neural networks

$$F_\theta(x) = L\overbrace{\left(f_{\theta^{(2)}}\left(f_{\theta^{(1)}}(x)\right)\right)}^{f_\theta}$$

$$\begin{aligned} F_\theta &: \mathbb{R}^d \times \mathbb{R}^n \to \mathbb{R} \\ f_{\theta^{(1)}} &: \mathbb{R}^d \times \mathbb{R}^{n'} \to \mathbb{R}^{d'} \\ f_{\theta^{(2)}} &: \mathbb{R}^{d'} \times \mathbb{R}^{n''} \to \mathbb{R}^{d''} \\ L &: \mathbb{R}^{d''} \to \mathbb{R} \end{aligned}$$

$$\begin{aligned} y &= F_\theta(x) \\ z^{(1)} &= f_{\theta^{(1)}}(x) \\ z^{(2)} &= f_{\theta^{(2)}}\left(z^{(1)}\right) \\ y &= L\left(z^{(2)}\right) \end{aligned}$$

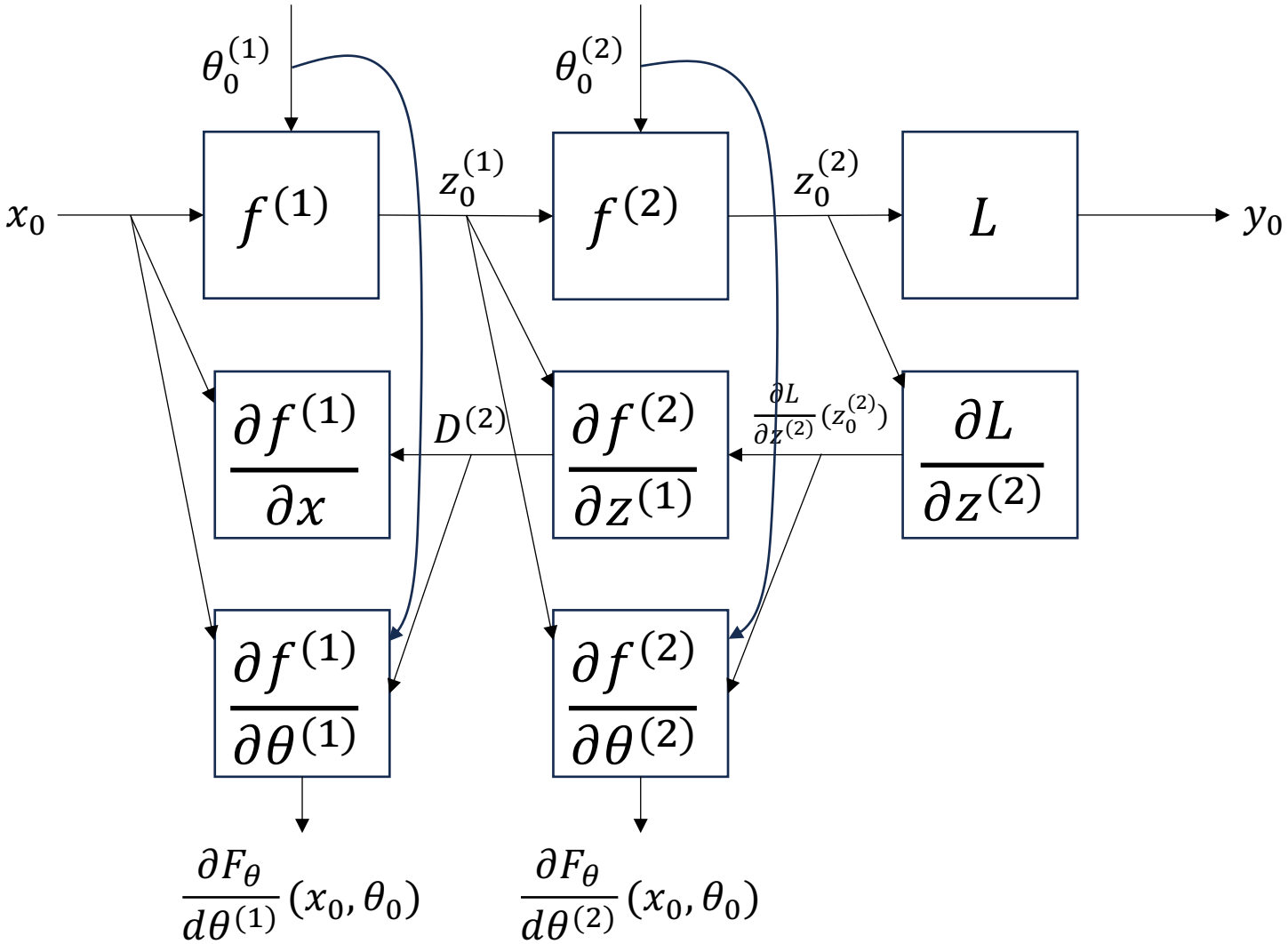
We need to compute $\frac{\partial F_\theta}{\partial \theta^{(1)}}$ and $\frac{\partial F_\theta}{\partial \theta^{(2)}}$ at specific inputs x_0 and θ_0

Backpropagation for neural networks

$$\frac{\partial F_\theta}{d\theta^{(1)}}(x_0, \theta_0) = \left(\underbrace{\left(\frac{\partial L}{\partial z^{(2)}}\left(z_0^{(2)}\right) \frac{\partial f^{(2)}}{\partial z^{(1)}}\left(z_0^{(1)}, \theta_0^{(2)}\right) \right)}_{D^{(2)}} \frac{\partial f^{(1)}}{\partial \theta^{(1)}}(x_0, \theta_0) \right)$$

$$\frac{\partial F_\theta}{d\theta^{(2)}}(x_0, \theta_0) = \left(\frac{\partial L}{\partial z^{(2)}}\left(z_0^{(2)}\right) \frac{\partial f^{(2)}}{\partial \theta^{(2)}}\left(z_0^{(1)}, \theta_0^{(2)}\right) \right)$$

Backpropagation for neural networks



Automatic Differentiation

- General version of Backpropagation Algorithm
- Two variants: reverse mode and forward mode
- Automatically calculate derivatives of computer programs
- See *Baydin et al., Automatic Differentiation in Machine Learning: a Survey, JMLR'18* for more

More resources

Chapter 2.5 of Dougal MacLaurin's thesis

Chapter 5 of book "Mathematics for Machine Learning"

DNN Frameworks

- Various frameworks such as PyTorch, TensorFlow, JAX have helped democratize deep learning

- Come with implementation of backpropagation, different variants of gradient descent, and various common layers
  - Only need gradients for new layers

- DNN architectures as a composition of layers
  - Represented as a graph to allow general compositions

PyTorch Basics

https://github.com/pytorch/examples/blob/main/mnist/main.py

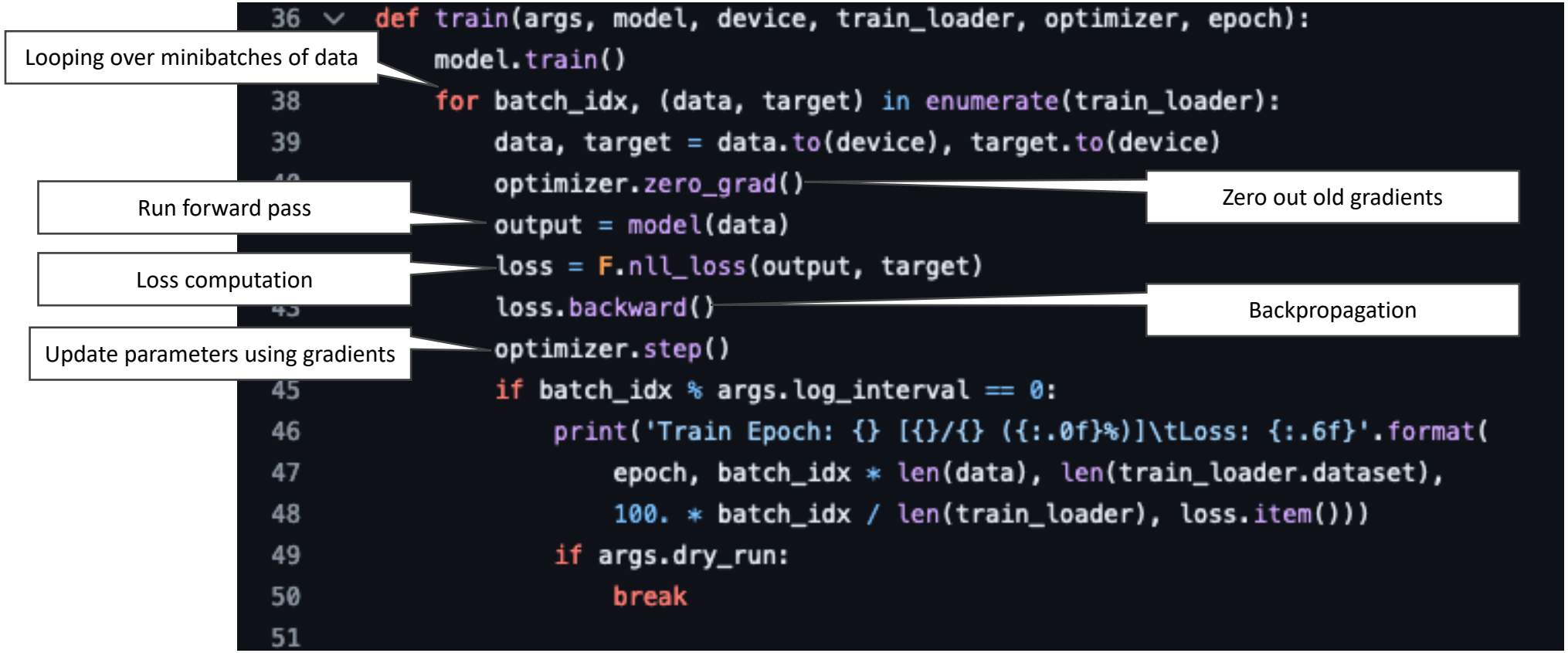
PyTorch Basics

Forward propagation

```
20      def forward(self, x):
21          x = self.conv1(x)
22          x = F.relu(x)
23          x = self.conv2(x)
24          x = F.relu(x)
25          x = F.max_pool2d(x, 2)
26          x = self.dropout1(x)
27          x = torch.flatten(x, 1)
28          x = self.fc1(x)
29          x = F.relu(x)
30          x = self.dropout2(x)
31          x = self.fc2(x)
32          output = F.log_softmax(x, dim=1)
33          return output
34
35
```

https://github.com/pytorch/examples/blob/main/mnist/main.py

PyTorch Basics




```
36  def train(args, model, device, train_loader, optimizer, epoch):
        model.train()
38      for batch_idx, (data, target) in enumerate(train_loader):
39          data, target = data.to(device), target.to(device)
            optimizer.zero_grad()
            output = model(data)
            loss = F.nll_loss(output, target)
            loss.backward()
            optimizer.step()
45          if batch_idx % args.log_interval == 0:
46              print('Train Epoch: {} [{}/{} ({:.0f}%)]\tLoss: {:.6f}'.format(
47                  epoch, batch_idx * len(data), len(train_loader.dataset),
48                  100. * batch_idx / len(train_loader), loss.item()))
49              if args.dry_run:
50                  break
51
```

https://github.com/pytorch/examples/blob/main/mnist/main.py